Two-Way Merge Sort

Data Structures Using C – Sorting Algorithms

Department of Computer Science and Engineering

Graphic Era (Deemed to be University), Dehradun

Recall: Sorting Algorithms

We have studied:

- Bubble Sort
- Insertion Sort
- Selection Sort

These algorithms generally require:

$$O(n^2)$$

time in the average and worst cases.

Question

Can we design a sorting algorithm that performs better for large datasets?

What is Merge Sort?

Merge Sort is a comparison-based sorting algorithm that follows the divide-and-conquer approach.

- Divide the array into two smaller subarrays.
- Recursively sort the subarrays.
- Merge the sorted subarrays.
- Continue until the complete array is sorted.

Main Idea

$$\boxed{\text{Divide} \rightarrow \text{Conquer} \rightarrow \text{Combine}}$$

What is Two-Way Merge Sort?

In Two-Way Merge Sort, an array is divided into two parts at each recursive level.

For an array of size n:

$$n \rightarrow \frac{n}{2} + \frac{n}{2}$$

Each half is further divided recursively.

Base Case

An array containing zero or one element is already sorted.

Divide-and-Conquer Strategy

1. Divide

Split the array into two approximately equal halves.

2. Conquer

Sort both halves recursively.

3. Combine

Merge the two sorted halves into one sorted array.

$$T(n) = 2T(n/2) + \Theta(n)$$

Merge Sort: Basic Example

Consider:

$$A = [8,\ 3,\ 5,\ 1]$$



Divide into two halves:

$$[8,\ 3] \qquad [5,\ 1]$$

Further divide:

$$[8] \quad [3] \quad [5] \quad [1]$$

Merge Sort: Combining the Elements

Individual elements are already sorted.

Merge:

$$[8] + [3] \rightarrow [3,\ 8]$$

$$[5] + [1] \rightarrow [1,\ 5]$$

Merge the two sorted arrays:

$$[3,\ 8] + [1,\ 5]$$

$$\boxed{[1,\ 3,\ 5,\ 8]}$$

Merge Sort: Complete Split and Merge

Divide Phase

$[8,\ 3,\ 5,\ 1]$



$[8,\ 3]$ $\quad$ $[5,\ 1]$

$[8]$ $\quad$ $[3]$ $\quad$ $[5]$ $\quad$ $[1]$

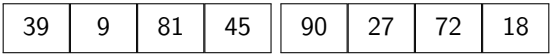
Merge Phase

$[3,\ 8]$ $\quad$ $[1,\ 5]$

Textbook-Style Example

Consider the array:

$$A = [39,\ 9,\ 81,\ 45,\ 90,\ 27,\ 72,\ 18]$$



- Number of elements: $n = 8$
- Sorting order: Ascending
- Strategy: Divide and Conquer

Divide Phase: Level 1

Initial array:

$$[39,\ 9,\ 81,\ 45,\ 90,\ 27,\ 72,\ 18]$$

Divide into two halves:

$$\underbrace{[39,\ 9,\ 81,\ 45]}_{\text{Left}}$$

$$\underbrace{[90,\ 27,\ 72,\ 18]}_{\text{Right}}$$

Each half contains four elements.

Divide Phase: Level 2

Left half:

$$[39,\ 9,\ 81,\ 45]$$

Divides into:

$$[39,\ 9] \qquad [81,\ 45]$$

Right half:

$$[90,\ 27,\ 72,\ 18]$$

Divides into:

$$[90,\ 27] \qquad [72,\ 18]$$

Divide Phase: Level 3

Divide every two-element subarray:

$$[39,\ 9] \rightarrow [39] + [9]$$

$$[81,\ 45] \rightarrow [81] + [45]$$

$$[90,\ 27] \rightarrow [90] + [27]$$

$$[72,\ 18] \rightarrow [72] + [18]$$

Base Case Reached

Every subarray now contains one element.
No further division is required.

Merge Phase: Level 1

Merge adjacent one-element arrays.

$$[39] + [9] \rightarrow [9,\ 39]$$

$$[81] + [45] \rightarrow [45,\ 81]$$

$$[90] + [27] \rightarrow [27,\ 90]$$

$$[72] + [18] \rightarrow [18,\ 72]$$

We now have four sorted subarrays.

Merge Phase: Level 2

Merge:

$$[9,\ 39] + [45,\ 81]$$

Result:

$$[9,\ 39,\ 45,\ 81]$$

Merge:

$$[27,\ 90] + [18,\ 72]$$

Result:

$$[18,\ 27,\ 72,\ 90]$$

Merge Phase: Final Level

Merge the two sorted halves:

$$L = [9,\ 39,\ 45,\ 81]$$

$$R = [18,\ 27,\ 72,\ 90]$$

Compare the first available element from each half.

Final result:

$$\boxed{[9,\ 18,\ 27,\ 39,\ 45,\ 72,\ 81,\ 90]}$$

Merge Sort: Complete Trace

Stage	Subarrays
Initial	39, 9, 81, 45, 90, 27, 72, 18
Divide 1	[39, 9, 81, 45] [90, 27, 72, 18]
Divide 2	[39, 9] [81, 45] [90, 27] [72, 18]
Divide 3	[39] [9] [81] [45] [90] [27] [72] [18]
Merge 1	[9, 39] [45, 81] [27, 90] [18, 72]
Merge 2	[9, 39, 45, 81] [18, 27, 72, 90]
Merge 3	[9, 18, 27, 39, 45, 72, 81, 90]

What is the Merge Operation?

The merge operation combines two sorted arrays into one sorted array.

Example:

$$L = [9,\ 39,\ 45,\ 81]$$

$$R = [18,\ 27,\ 72,\ 90]$$

The result must contain all elements in sorted order.

$$M = [9,\ 18,\ 27,\ 39,\ 45,\ 72,\ 81,\ 90]$$

Merge Operation: Three Indices

During merging, we use:

Index	Purpose
i	Current element in left subarray
j	Current element in right subarray
k	Current position in temporary array

Initially:

$$i = 0, \qquad j = 0, \qquad k = 0$$

Compare:

$$L[i] \quad \text{and} \quad R[j]$$

Copy the smaller value to the temporary array.

Merge Operation: First Comparison

$$L = [9,\ 39,\ 45,\ 81]$$

$$R = [18,\ 27,\ 72,\ 90]$$

Initially:

$$i = 0, \quad j = 0$$

Compare:

$$9 < 18$$

Copy 9.

Temporary array:

$$TEMP = [9]$$

Now:

Merge Operation: Second Comparison

Current elements:

$$L[i] = 39$$

$$R[j] = 18$$

Since:

$$18 < 39$$

copy 18.

Temporary array:

$$TEMP = [9,\ 18]$$

Now:

$$i = 1, \quad j = 1$$

Merge Operation: Third Comparison

Current elements:

$$L[i] = 39$$

$$R[j] = 27$$

Since:

$$27 < 39$$

copy 27.

Temporary array:

$$TEMP = [9,\ 18,\ 27]$$

Now:

$$i = 1, \quad j = 2$$

Merge Operation: Remaining Comparisons

Continue comparing:

$$39 < 72 \Rightarrow \text{Copy } 39$$

$$45 < 72 \Rightarrow \text{Copy } 45$$

$$72 < 81 \Rightarrow \text{Copy } 72$$

$$81 < 90 \Rightarrow \text{Copy } 81$$

The left subarray is exhausted.
Copy the remaining element 90.

$$\boxed{TEMP = [9,\ 18,\ 27,\ 39,\ 45,\ 72,\ 81,\ 90]}$$

Merge Operation: Complete Trace

Step	Selected	Temporary Array
1	9	9
2	18	9, 18
3	27	9, 18, 27
4	39	9, 18, 27, 39
5	45	9, 18, 27, 39, 45
6	72	9, 18, 27, 39, 45, 72
7	81	9, 18, 27, 39, 45, 72, 81
8	90	9, 18, 27, 39, 45, 72, 81, 90

Merge Operation: Algorithm

MERGE(A, left, mid, right)

1. Set $i = left$, $j = mid + 1$, $k = left$.
2. While $i \leq mid$ and $j \leq right$:
   - If $A[i] \leq A[j]$, copy $A[i]$ to $TEMP[k]$.
   - Otherwise, copy $A[j]$ to $TEMP[k]$.
   - Advance the corresponding index and k.
3. Copy remaining elements of the left half.
4. Copy remaining elements of the right half.
5. Copy $TEMP[left \ldots right]$ back into A.

Merge Sort: Recursive Algorithm

MERGE_SORT(A, left, right)

1. If $left < right$:
   1. Calculate:
      $$mid = left + \frac{right - left}{2}$$
      using integer division.
   2. Call:
      $$MERGE_SORT(A, left, mid)$$
   3. Call:
      $$MERGE_SORT(A, mid + 1, right)$$
   4. Call:
      $$MERGE(A, left, mid, right)$$
2. Return.

Merge Function in C: Part 1

```c
void merge(int a[], int temp[],
           int left, int mid, int right)
{
    int i = left;
    int j = mid + 1;
    int k = left;

    while (i <= mid && j <= right)
    {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    // Remaining elements are copied next.
```

Merge Function in C: Part 2

```
    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= right)
        temp[k++] = a[j++];

    for (int p = left; p <= right; p++)
        a[p] = temp[p];
}
```

Important

The temporary array stores the merged values.
These values are then copied back to the original array.

Recursive Merge Sort Function

```
1  void mergeSort(int a[], int temp[],
2                 int left, int right)
3  {
4      if (left >= right)
5          return;
6
7      int mid = left + (right - left) / 2;
8
9      mergeSort(a, temp, left, mid);
10
11     mergeSort(a, temp, mid + 1, right);
12
13     merge(a, temp, left, mid, right);
14 }
```

Base Case

When $left \geq right$, the subarray has at most one element and requires no sorting.

Complete C Program: Part 1

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void merge(int a[], int temp[],
5             int left, int mid, int right)
6  {
7      int i = left, j = mid + 1, k = left;
8
9      while (i <= mid && j <= right)
10     {
11         if (a[i] <= a[j])
12             temp[k++] = a[i++];
13         else
14             temp[k++] = a[j++];
15     }
16
17     while (i <= mid)
18         temp[k++] = a[i++];
19
20     while (j <= right)
21         temp[k++] = a[j++];
22
23     for (int p = left; p <= right; p++)
24         a[p] = temp[p];
25 }
```

Complete C Program: Part 2

```c
void mergeSort(int a[], int temp[],
               int left, int right)
{
    if (left >= right)
        return;

    int mid = left + (right - left) / 2;

    mergeSort(a, temp, left, mid);
    mergeSort(a, temp, mid + 1, right);
    merge(a, temp, left, mid, right);
}

int main(void)
{
    int a[] = {39, 9, 81, 45, 90, 27, 72, 18};
    int n = sizeof(a) / sizeof(a[0]);

    int *temp = malloc(n * sizeof *temp);

    if (temp == NULL)
        return 1;

    mergeSort(a, temp, 0, n - 1);

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
    free(temp);
    return 0;
}
```

Merge Sort: Expected Output

Input:

```
39 9 81 45 90 27 72 18
```

Output:

```
9 18 27 39 45 72 81 90
```

Observation

The recursive calls divide the array.
The merge operation combines the sorted subarrays.

Merge Sort: Recurrence Relation

At each recursive step:

- The problem is divided into two halves.
- Each half is sorted recursively.
- Merging requires linear time.

Therefore:

$$T(n) = 2T(n/2) + \Theta(n)$$

For the base case:

$$T(1) = \Theta(1)$$

This recurrence gives:

$$\boxed{T(n) = \Theta(n \log n)}$$

Merge Sort: Recursion Levels

At each level, the subarray size is approximately halved.

$$n, \quad \frac{n}{2}, \quad \frac{n}{4}, \quad \frac{n}{8}, \ldots, 1$$

After k divisions:

$$\frac{n}{2^k} = 1$$

Therefore:

$$2^k = n$$

$$\boxed{k = \log_2 n}$$

The recursion depth is:

$$O(\log n)$$

Merge Sort: Work at Each Level

For $n = 8$:

Merge Level	Subarrays Merged	Elements Processed
1	4 pairs of size 1	8
2	2 pairs of size 2	8
3	1 pair of size 4	8

Work per merge level:

$$\Theta(n)$$

Number of merge levels:

$$\Theta(\log n)$$

Total:

$$\boxed{\Theta(n \log n)}$$

Merge Sort: Time Complexity

Case	Time Complexity
Best	$\Theta(n \log n)$
Average	$\Theta(n \log n)$
Worst	$\Theta(n \log n)$

Important

Merge Sort has the same asymptotic running time in all three cases for the standard top-down implementation.

Merge Sort: Space Complexity

Merge Sort requires additional storage for merging.
For an array containing n elements:

$$\text{Temporary array space} = O(n)$$

Recursive calls require:

$$O(\log n)$$

stack space.

Overall auxiliary space:

$$\boxed{O(n)}$$

Observation

Standard array-based Merge Sort is not in-place because it requires a temporary array.

Why is Merge Sort Stable?

Consider:

$$[(4, A),\ (2, B),\ (4, C)]$$

The number represents the sorting key.

After stable sorting:

$$[(2, B),\ (4, A),\ (4, C)]$$

The relative order of equal keys remains unchanged.

Important Condition

During merging, choose the element from the left subarray when keys are equal:

$$A[i] \leq A[j]$$

This preserves stability.

Properties of Merge Sort

Property	Merge Sort
Comparison-based	Yes
Divide-and-conquer	Yes
Stable	Yes, with stable merging
In-place	No, standard array version
Adaptive	No, standard top-down version
Best time	$\Theta(n \log n)$
Worst time	$\Theta(n \log n)$
Auxiliary space	$O(n)$

Advantages and Limitations

Advantages

- Predictable $O(n \log n)$ time.
- Stable sorting.
- Suitable for large datasets.
- Useful for linked lists.
- Useful in external sorting.

Limitations

- Requires extra memory for arrays.
- Recursive implementation.
- More overhead for very small arrays.
- Standard version does not exploit already sorted input.

Comparison with Earlier Algorithms

Property	Bubble*	Insertion	Selection	Merge
Best	$O(n)$	$O(n)$	$O(n^2)$	$O(n \log n)$
Average	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$
Worst	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$
Space	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Stable	Yes	Yes	No	Yes

* Optimized Bubble Sort.

Merge Sort improves asymptotic time complexity at the cost of additional memory.

Classroom Activity 1

Apply Merge Sort to:

$$[38,\ 27,\ 43,\ 3,\ 9,\ 82,\ 10]$$

Tasks:

1. Draw the recursive division.
2. Identify the base cases.
3. Merge adjacent sorted subarrays.
4. Show the final sorted array.

Classroom Activity 1: Solution

Initial:

$$[38,\ 27,\ 43,\ 3,\ 9,\ 82,\ 10]$$

Divide:

$$[38,\ 27,\ 43,\ 3] \quad [9,\ 82,\ 10]$$

Then:

$$[38,\ 27]\ [43,\ 3]\ [9,\ 82]\ [10]$$

Merge:

$$[27,\ 38]\ [3,\ 43]\ [9,\ 82]\ [10]$$

Then:

$$[3,\ 27,\ 38,\ 43] \quad [9,\ 10,\ 82]$$

Final:

$$\boxed{[3,\ 9,\ 10,\ 27,\ 38,\ 43,\ 82]}$$

Classroom Activity 2: Merge Operation

Two sorted arrays are given:

$$L = [2,\ 8,\ 15,\ 20]$$

$$R = [1,\ 7,\ 12,\ 25]$$

Perform the merge operation.
Show:

1. Current values of i and j.
2. Selected element.
3. Updated temporary array.

Classroom Activity 2: Solution

Step	Temporary Array
1	1
2	1, 2
3	1, 2, 7
4	1, 2, 7, 8
5	1, 2, 7, 8, 12
6	1, 2, 7, 8, 12, 15
7	1, 2, 7, 8, 12, 15, 20
8	1, 2, 7, 8, 12, 15, 20, 25

MCQ 1

Which design technique is used by Merge Sort?

1. Greedy approach
2. Divide and conquer
3. Dynamic programming
4. Backtracking

MCQ 1

Which design technique is used by Merge Sort?

1. Greedy approach
2. Divide and conquer
3. Dynamic programming
4. Backtracking

Answer

Option 2: Divide and conquer.

MCQ 2

What is the worst-case time complexity of Merge Sort?

1. $O(n)$
2. $O(n^2)$
3. $O(n \log n)$
4. $O(\log n)$

MCQ 2

What is the worst-case time complexity of Merge Sort?

1. $O(n)$
2. $O(n^2)$
3. $O(n \log n)$
4. $O(\log n)$

Answer

Option 3: $O(n \log n)$.

MCQ 3

What is the auxiliary space complexity of standard array-based Merge Sort?

1. $O(1)$
2. $O(\log n)$
3. $O(n)$
4. $O(n^2)$

MCQ 3

What is the auxiliary space complexity of standard array-based Merge Sort?

1. $O(1)$
2. $O(\log n)$
3. $O(n)$
4. $O(n^2)$

Answer

Option 3: $O(n)$.

MCQ 4

Which comparison helps preserve stability during merging?

1. $A[i] < A[j]$
2. $A[i] \leq A[j]$
3. $A[i] > A[j]$
4. $A[i] \neq A[j]$

MCQ 4

Which comparison helps preserve stability during merging?

1. $A[i] < A[j]$
2. $A[i] \leq A[j]$
3. $A[i] > A[j]$
4. $A[i] \neq A[j]$

Answer

Option 2: $A[i] \leq A[j]$.

Common Programming Mistakes

1. Incorrect midpoint calculation.
2. Missing recursion base case.
3. Incorrect left and right boundaries.
4. Forgetting to copy remaining elements.
5. Copying temporary values to wrong indices.
6. Using incorrect comparison for stable merging.
7. Allocating insufficient temporary memory.

Important

Array indices must remain within valid bounds during recursive splitting and merging.

Practice Questions

1. Explain the divide-and-conquer strategy.
2. Explain Two-Way Merge Sort with an example.
3. Write the Merge Sort algorithm.
4. Write the algorithm for merging two sorted arrays.
5. Implement Merge Sort in C.
6. Derive the recurrence relation for Merge Sort.
7. Explain its time and space complexities.
8. Why is Merge Sort stable?
9. Compare Merge Sort with Insertion Sort.

Lecture Summary

Key Concepts

- Merge Sort follows divide and conquer.
- The array is divided into two halves recursively.
- Sorted halves are merged.
- Recurrence:

$$T(n) = 2T(n/2) + \Theta(n)$$

- Time complexity:

$$\Theta(n \log n)$$

- Auxiliary space:

$$O(n)$$

References

1. Reema Thareja, *Data Structures Using C*, chapter on Searching and Sorting.
2. Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.
3. Thomas H. Cormen et al., *Introduction to Algorithms*.

Lecture Feedback

Your Feedback Matters

Please share your feedback on:

- Divide-and-conquer strategy
- Recursive splitting
- Merge operation
- C implementation
- Complexity analysis



Scan to Submit Feedback

Thank You

Questions and Discussion

Divide $\rightarrow$ Conquer $\rightarrow$ Combine